SPEECH RECOGNITION · LECTURE 1 / 6

From sound to statistical models

Features, acoustic models and language models,
each measured on its own before we combine them this afternoon.

Data Science Project · Master ISD · Université Paris-Saclay · 10 Nov

Four days, one pipeline improved step by step

Day	Content	Deliverable
1 · 10 Nov	Classical ASR: MFCC, GMM-HMM, n-grams, Viterbi	d1
2 · 17 Nov	Neural ASR (CTC, Whisper) + external language models	d2
3 · 24 Nov	Decide what to deploy; live push-to-talk demo	d3
Final	Defence of your decisions	half of the grade

Every day you

- implement the `TODO` stubs: each one is tied to an equation or algorithm on a slide (see the lab slide),
- benchmark, and log one row per experiment to `results/results.jsonl`,
- write every decision, with its evidence, in `DECISIONS.md`.

Today: the pieces of a recogniser, each measured alone

Plan

1. The decision rule; comparing systems
2. Features: waveform $\rightarrow \boldsymbol{x}_t \in \mathbb{R}^{39}$
3. Acoustic model: one GMM-HMM per word
4. Language model: n-grams, perplexity

Data: Free Spoken Digit Dataset (FSDD), 6 speakers $\times$ 10 digits $\times$ 50 repetitions, $f_s = 8$ kHz. Repetitions 0–4 are the **300 test files**.

You implement this morning

TODO `pareto_mask_array` slide 7
TODO `recognise_isolated` slides 17, 21
TODO `NGramLM.fit / prob / perplexity` 23–28

Provided (read it)

`mfcc_features`, `Normaliser`, `train_word_models` (Baum–Welch, `hmmlearn`), `degrade`, `common/bench.py`

Part 1

The decision rule, and how we compare systems

Speech recognition as a decision problem

Given the acoustic observation $X = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$, choose the most probable word sequence $W = (w_1, \ldots, w_L)$:

$$\hat{W} = \arg\max_W P(W \mid X) \overset{\text{Bayes}}{=} \arg\max_W \frac{p(X \mid W)\, P(W)}{p(X)}$$

$$= \arg\max_W \underbrace{p(X \mid W)}_{\text{acoustic model}} \cdot \underbrace{P(W)}_{\text{language model}}$$

The last step holds because $p(X)$ does not depend on W.

Features (part 2)

How X is computed from the waveform.

Acoustic model (part 3)

$p(X \mid W)$: one HMM per word, chained for W.

Language model (part 4)

$P(W)$: trained on text only.

This morning: each piece is measured alone. This afternoon (L2): the search for $\hat{W}$ over all word sequences.

We compare systems with Pareto fronts, not a single score

Each system $\mathcal{M}$ (a configuration of the pipeline) is measured on several objectives $f_o(\mathcal{M})$, $o \in \mathcal{O}$ (e.g. error rate, latency). Write every objective as *lower is better* (use $-$accuracy for accuracy).

Definitions

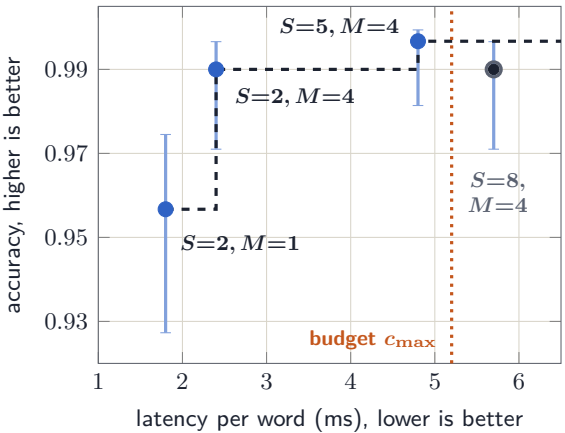
- **Dominance.** $\mathcal{M}$ dominates $\mathcal{M}'$ (written $\mathcal{M} \succ \mathcal{M}'$) iff

$$f_o(\mathcal{M}) \le f_o(\mathcal{M}') \text{ for all } o \in \mathcal{O} \quad \text{and} \quad f_o(\mathcal{M}) < f_o(\mathcal{M}') \text{ for at least one } o.$$

- **Pareto front.** The set of systems that no other system dominates.
- **Budget.** A constraint on a cost, e.g. latency $\le c_{\max}$. Your choice minimises the error under it: $\mathcal{M}^\star \in \underset{\mathcal{M}\,:\,\mathrm{cost}(\mathcal{M}) \le c_{\max}}{\arg\min} \mathrm{err}(\mathcal{M})$.

The cheapest minimiser is on the front: a system dominating it would also meet the budget and be at least as good.

Example: the isolated-word front of this morning



- Dashed: the front. $S{=}8$ is dominated by $S{=}5$ (S states, M Gaussians per state).
- Bars: 95 % Wilson interval for an accuracy $\hat{p}$ measured on $N_{\text{test}} = 300$ files (N for short), with $z = 1.96$ the 97.5 % Gaussian quantile:

$$\frac{\hat{p} + \frac{z^2}{2N} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + z^2/N}$$

- One error $= 0.33$ point: the three best systems are not separable on 300 files.

`pareto_mask_array`: what to implement

TODO `common/pareto.py`

`pareto_mask_array(values, directions)`
In: `values` $[N_{\text{sys}} \times |\mathcal{O}|]$; `directions`: `"min"`/`"max"` per column.
Out: boolean mask $[N_{\text{sys}}]$, `True` = on the front.

negate every `"max"` column ▷ all lower is better
for each row (system) $\mathcal{M}$ **do**
    $\text{keep}[\mathcal{M}] \leftarrow$ **true**
    for each row $\mathcal{M}'$ **do**
        if $f(\mathcal{M}') \leq f(\mathcal{M})$ on all columns **and** $<$ on one **then**
            $\text{keep}[\mathcal{M}] \leftarrow$ **false**
return keep

- Cost $O(N_{\text{sys}}^2 |\mathcal{O}|)$: fine for a few hundred configurations. The inner loop is one vectorised NumPy line.
- Ties: two identical points do *not* dominate each other; both stay.
- Provided on top of it: `pareto_mask(df, {col: dir})` (drops NaN rows) and `plot_pareto` (front + budget line).

Measuring latency correctly is half the work

Protocol

- **Warm up**: the first calls pay lazy initialisation and caches; discard them.
- **Synchronise**: GPU calls are asynchronous; stop the clock after `torch.cuda.synchronize()`.
- **Same conditions**: same data, machine and code path for every system.

Metrics

Latencies of R requests (one per utterance):

- p50, p90: 50th and 90th percentiles of the R latencies (`np.percentile`, linear interpolation). Users feel the tail, not the mean.
- Real-time factor $\mathrm{RTF} = \dfrac{\text{compute time}}{\text{audio duration}}$; $\mathrm{RTF} < 1$: the system keeps up with speech.

The provided harness `common/bench.py` implements this protocol; read it before writing your own timing code.

Part 2

Features: from waveform to x_t

MFCC: a compact, roughly decorrelated description of each 10 ms



- 100 vectors per second of speech: a one-second digit is a 100×39 matrix.
- Thrown away: phase, pitch harmonics (fine spectral structure), everything above the 13th cepstral coefficient.
- The DCT *roughly decorrelates* the 40 log-mel energies: this justifies *diagonal* covariances in the GMMs (part 3).
- Provided: `mfcc_features(y, sr)` (`librosa`); equations on the next slide.

MFCC, step by step (1/2), as computed by `mfcc_features`

1. Framing. Window $N_w = 0.025\, f_\mathrm{s}$ (200 samples), hop $N_h = 0.010\, f_\mathrm{s}$ (80), Hann window; frames are *centred* (zero padding at the edges). For $t = 1, \ldots, T$ and $0 \le \tau < N_w$:

$$y_t[\tau] = y\big[(t{-}1)N_h - \tfrac{N_w}{2} + \tau\big]\,\mathrm{win}[\tau], \qquad \mathrm{win}[\tau] = \tfrac{1}{2} - \tfrac{1}{2}\cos\tfrac{2\pi\tau}{N_w}.$$

2. Power spectrum.

$$P_t[f] = \big|\mathrm{DFT}(y_t)[f]\big|^2, \quad f = 0, \ldots, N_w/2.$$

3. Mel filterbank. $N_\mathrm{mel} = 40$ triangular filters H_ν, $\nu = 1, \ldots, N_\mathrm{mel}$, equally spaced on the mel scale (linear below 1 kHz, log above), 0 to $f_\mathrm{s}/2$:

$$E_t[\nu] = \textstyle\sum_f H_\nu[f]\, P_t[f].$$

4. Log (dB, floored at the utterance maximum -80 dB):

$$\tilde{E}_t[\nu] = 10 \log_{10} \max(E_t[\nu], 10^{-10}).$$

MFCC, step by step (2/2)

5. DCT-II (orthonormal), keep $\ell = 0, \dots, 12$:

$$c_t[\ell] = \rho_\ell \sum_{\nu=1}^{N_{\text{mel}}} \tilde{E}_t[\nu] \cos\left(\pi \ell \, (\nu - \tfrac{1}{2}) / N_{\text{mel}}\right),$$

$\rho_0 = \sqrt{1/N_{\text{mel}}}$, $\rho_\ell = \sqrt{2/N_{\text{mel}}}$ for $\ell > 0$.

6. Dynamics over ± 4 frames. First derivative (linear regression):

$$\Delta c_t = \frac{\sum_{p=1}^{4} p \, (c_{t+p} - c_{t-p})}{2 \sum_{p=1}^{4} p^2};$$

$\Delta^2 c_t$: second derivative of a quadratic fitted by least squares on $c_{t-4}, \dots, c_{t+4}$ (not Δ applied twice).

7. Stack. $\boldsymbol{x}_t = (c_t, \ \Delta c_t, \ \Delta^2 c_t) \in \mathbb{R}^D$, $D = 39$.

Textbook vs our code (`librosa` defaults)

No pre-emphasis ($y[\tau] - 0.97\, y[\tau - 1]$ in textbooks); Hann, not Hamming; Slaney mel filters (area-normalised); dB with an 80 dB floor; centred frames.

Every MFCC step is a modelling choice you can probe

Step	Why	What to try
Mel scale	frequency resolution of hearing	linear filterbank
Log	loudness $\approx$ log energy; a channel filter becomes additive	no log
DCT, 13 coeff.	keep the spectral envelope, drop pitch	20 coefficients
Δ, Δ^2	dynamics: HMM frames are independent given the state	static only
Normalisation	remove channel and level effects	per utterance vs global

Normalisation in the provided code (`Normaliser`)

Mean $\hat{\boldsymbol{\mu}}$ and standard deviation $\hat{\boldsymbol{\sigma}}$ over *all training frames* (words and silence); then every frame

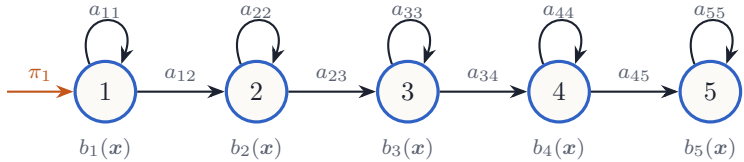
$$\boldsymbol{x}_t \leftarrow (\boldsymbol{x}_t - \hat{\boldsymbol{\mu}}) \oslash \hat{\boldsymbol{\sigma}} \qquad (\oslash : \text{ element-wise division}).$$

Pitfall we hit: per-utterance normalisation on isolated training words, then on long test utterances, gives different statistics. Use one global normaliser.

Part 3

Acoustic model: one GMM-HMM per word

One left-to-right HMM per word



Model of word w: $\theta_w = (\boldsymbol{\pi}, A, \{b_j\})$, S states.

- $q_t \in \{1, \ldots, S\}$: hidden state at frame t.
- $\pi_j = P(q_1 = j)$; here $\boldsymbol{\pi} = (1, 0, \ldots, 0)$.
- $a_{ij} = P(q_t = j \mid q_{t-1} = i)$: only a_{jj} (stay) and $a_{j,j+1}$ (move) are non-zero; initialised to $0.6/0.4$, and $a_{SS} = 1$.
- $b_j(\boldsymbol{x}) = p(\boldsymbol{x}_t = \boldsymbol{x} \mid q_t = j)$: a GMM (next slide).

Topology is a prior

Speech moves forward in time: no backward arcs. Training (EM, *Expectation–Maximisation*, slides 18–20) keeps zero transitions at zero: the topology survives.

Durations

Staying κ frames in state j: $P(\kappa) = a_{jj}^{\kappa-1}(1 - a_{jj})$, mean $1/(1 - a_{jj})$. A 0.4 s digit over 5 states: 8 frames per state, $a_{jj} \approx 0.875$.

Each state emits feature vectors from a Gaussian mixture

$$b_j(\boldsymbol{x}) = \sum_{m=1}^{M} \omega_{jm}\, \mathcal{N}\big(\boldsymbol{x};\ \boldsymbol{\mu}_{jm},\ \operatorname{diag}\boldsymbol{\sigma}^2_{jm}\big), \qquad \omega_{jm} \geq 0,\ \ \textstyle\sum_m \omega_{jm} = 1.$$

With a diagonal covariance, the log-density is a sum over the D dimensions:

$$\log \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}, \operatorname{diag}\boldsymbol{\sigma}^2) = -\frac{1}{2} \sum_{d=1}^{D} \Big[\log(2\pi\sigma_d^2) + \frac{(x_d - \mu_d)^2}{\sigma_d^2} \Big].$$

In the log domain, the mixture is a *log-sum-exp* (next slide):

$$\log b_j(\boldsymbol{x}) = \operatorname{logsumexp}_m \big[\log \omega_{jm} + \log \mathcal{N}(\boldsymbol{x}; \boldsymbol{\mu}_{jm}, \operatorname{diag}\boldsymbol{\sigma}^2_{jm}) \big].$$

Mixtures	**Diagonal covariances**	**Model size (`n_params`)**
Several speakers and contexts per state: one Gaussian is too narrow.	$2D$ instead of $D + D(D{+}1)/2$ parameters per Gaussian; justified by the DCT.	Stored values: $S\,M(2D{+}1) + S^2$ per word (1 605 for $S{=}5$, $M{=}4$); fewer are free.

Adding probabilities stored as logs: the log-sum-exp trick

Why logs. A 39-dimensional frame has $\log b_j(\boldsymbol{x}_t) \approx -40$, so one second (100 frames) gives $p \approx e^{-4000}$. The smallest positive double is $\approx e^{-745}$: the product underflows to 0. We store $\log p$; products become sums.

The problem. Mixtures, the forward algorithm and EM also *add* probabilities, $\log \sum_m e^{\xi_m}$; computing each e^{ξ_m} underflows again.

The trick: factor out the largest term

$$\operatorname{logsumexp}_m \xi_m = \xi^\star + \log \sum_m e^{\xi_m - \xi^\star}, \quad \xi^\star = \max_m \xi_m.$$

Exact, not an approximation. The largest term becomes $e^0 = 1$, so the sum lies in $[1, M]$: no underflow, no overflow. If every $\xi_m = -\infty$ (no possible path), return $-\infty$.

Example: $\xi = (-1000, -1001, -1003)$

Naive: e^{-1000} is 0 in double precision, so $\log(0 + 0 + 0) = -\infty$.

Trick: $-1000 + \log(1 + e^{-1} + e^{-3})$
$= -1000 + 0.349 = -999.651$.

Where you meet it

The GMM mixture (previous slide), the forward recursion (slide 17), EM's posteriors (slide 19). In code: `scipy.special.logsumexp`, `np.logaddexp`. Viterbi (L2) takes a $\max$ instead of a sum: no trick needed.

The likelihood of a word model sums over all state paths

For one state path $q = (q_1, \ldots, q_T)$, frames are independent *given the states*:

$$p(X, q \mid \theta) = \pi_{q_1}\, b_{q_1}(\boldsymbol{x}_1) \prod_{t=2}^{T} a_{q_{t-1} q_t}\, b_{q_t}(\boldsymbol{x}_t).$$

The likelihood of the word marginalises the hidden path:

$$p(X \mid \theta) = \sum_{q \in \{1,\ldots,S\}^T} p(X, q \mid \theta) \qquad (S^T \text{ terms: } 5^{100} \text{ for one second}).$$

Three classical problems, three algorithms

Problem	Quantity	Algorithm	When
Evaluation	$p(X \mid \theta) = \sum_q p(X, q \mid \theta)$	forward, $O(TS^2)$	this morning
Decoding	$\hat{q} = \arg\max_q p(X, q \mid \theta)$	Viterbi (max for sum)	this afternoon
Training	$\max_\theta \prod_{\text{train}} p(X \mid \theta)$	Baum–Welch (EM)	provided

The forward algorithm computes $p(X \mid \theta)$ in $O(TS^2)$

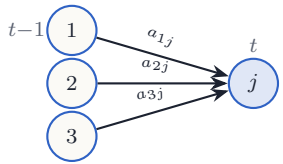
$\alpha_t(j) = p(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_t,\ q_t = j \mid \theta)$: total probability of all paths that are in state j at frame t.

Recursion

$$\alpha_1(j) = \pi_j\, b_j(\boldsymbol{x}_1)$$

$$\alpha_t(j) = b_j(\boldsymbol{x}_t) \sum_{i=1}^{S} \alpha_{t-1}(i)\, a_{ij}, \qquad t = 2, \ldots, T$$

$$p(X \mid \theta) = \sum_{j=1}^{S} \alpha_T(j)$$



- **Why logs**, and how to add log-probabilities: slide 15.
- `GMMHMM.score(X)` (`hmmlearn`) returns $\log p(X \mid \theta)$, computed exactly this way.

The same recursion in the log domain (what is actually computed)

$$\log \alpha_t(j) = \log b_j(\boldsymbol{x}_t) + \operatorname{logsumexp}_i \left[\log \alpha_{t-1}(i) + \log a_{ij} \right], \qquad \log 0 = -\infty.$$

EM (Expectation–Maximisation): learning with hidden variables

Training wants $\hat{\theta} = \arg\max_\theta \sum_{\text{train}} \log p(X \mid \theta)$, where $\log p(X \mid \theta) = \log \sum_q p(X, q \mid \theta)$: the sum over hidden paths sits *inside* the log, so there is no closed-form solution. If we *saw* the hidden variables (which state, which Gaussian produced each frame), training would be counting and averaging. EM uses this, iteratively.

One EM iteration, from θ^{old}

E-step. With θ^{old}, compute the *posterior* of the hidden variables, e.g. $\zeta_t(j) = P(q_t{=}j \mid X, \theta^{\text{old}})$: how much frame t belongs to state j.

M-step. Re-estimate θ^{new} as if the hidden variables were observed, each frame counted with weight ζ: weighted counts and weighted averages, in closed form.

What EM guarantees, and what it does not

- An iteration never lowers the training likelihood.
- It stops at a local optimum: the result depends on the initialisation.
- A parameter at 0 gets no posterior weight: it stays 0.
- Cost: one pass over the training data per iteration (`n_iter`).

EM on one GMM: every frame is shared between the Gaussians

One state, M Gaussians; the hidden variable of frame t is the component that produced $\boldsymbol{x}_t$. Sums run over all training frames.

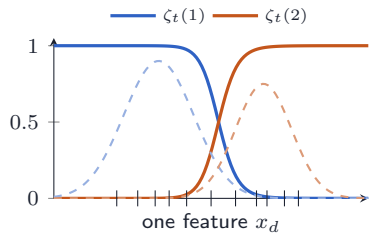
E-step: the *responsibility* of component m for frame t (Bayes' rule; log-sum-exp in the denominator):

$$\zeta_t(m) = \frac{\omega_m \, \mathcal{N}(\boldsymbol{x}_t; \boldsymbol{\mu}_m, \operatorname{diag} \boldsymbol{\sigma}_m^2)}{\sum_{m'} \omega_{m'} \, \mathcal{N}(\boldsymbol{x}_t; \boldsymbol{\mu}_{m'}, \operatorname{diag} \boldsymbol{\sigma}_{m'}^2)}.$$

M-step: weighted counts and averages,

$$\omega_m \leftarrow \frac{\sum_t \zeta_t(m)}{\sum_t 1}, \qquad \boldsymbol{\mu}_m \leftarrow \frac{\sum_t \zeta_t(m) \, \boldsymbol{x}_t}{\sum_t \zeta_t(m)},$$

$$\boldsymbol{\sigma}_m^2 \leftarrow \frac{\sum_t \zeta_t(m) \, (\boldsymbol{x}_t - \boldsymbol{\mu}_m)^2}{\sum_t \zeta_t(m)} \quad \text{(element-wise)}.$$



Dashed: the two Gaussians; ticks: frames. A frame between them counts partly for both.

Hard assignments ($\zeta_t(m) \in \{0, 1\}$, nearest mean) give **k-means**. In a GMM-HMM the hidden variable is the pair (state, component): next slide.

Training (provided): Baum–Welch is EM for HMMs; a flat start

Each word model is trained on the recordings of that word only.

- **E-step**: the forward pass and a symmetric backward pass give the posteriors $\zeta_t(j) = P(q_t{=}j \mid X, \theta)$ and $\zeta_t(i,j) = P(q_{t-1}{=}i, q_t{=}j \mid X, \theta)$.
- **M-step**: soft counts, e.g.

$$a_{ij} \leftarrow \frac{\sum_{t\geq 2} \zeta_t(i,j)}{\sum_{t\geq 2} \zeta_{t-1}(i)};$$

  each state's GMM as on slide 19, frames weighted by the posterior of (state j, component m).
- A zero a_{ij} gets zero posterior mass: it stays zero.

Initialisation decides the result

EM reaches a *local* optimum.

- **Flat start** (provided): cut every training word into S equal chunks; fit one GMM on the frames of each chunk.
- Default k-means on pooled frames left empty components and produced NaNs.

Silence model

`sil`: 3 states, ≤ 2 Gaussians, trained on recording edges and low-level noise; used in L2.

Isolated words: score every word model, keep the best

One recording = one word, all words equally likely ($P(w) = 1/V$), so the decision rule becomes

$$\hat{w} = \arg\max_{w \in \mathcal{V}} \ \log p(X \mid \theta_w) \qquad (V = 10 \text{ forward passes}).$$

TODO `recognise_isolated(wm, feats, vocab)` **in** `speech/gmm_hmm.py`

`wm.models[w]` is the trained `GMMHMM` of word w; `feats` $[T \times D]$ are normalised features. Return the word whose model gives the highest `.score(feats)`.

Configuration	Accuracy (clean) [95 % interval]	Latency / word	
$S = 2$ states, $M = 1$	0.957 [0.927, 0.975]	1.8 ms	fast, coarse
$S = 2$, $M = 4$	0.990 [0.971, 0.997]	2.4 ms	front
$S = 5$, $M = 4$	0.997 [0.981, 0.999]	4.8 ms	front
$S = 8$, $M = 4$	0.990 [0.971, 0.997]	5.7 ms	dominated

Acoustic degradations: what an acoustic-only model cannot handle

Condition	Simulation (`speech/degrade.py`, provided)	Acc.
Clean	—	0.983
White noise, 10 dB	$y + g\,\epsilon$, $\epsilon \sim \mathcal{N}(0,1)$; gain g such that $10\log_{10}\frac{\overline{y^2}}{g^2\,\overline{\epsilon^2}} = \mathrm{SNR}$	0.717
Babble, 5 dB	same mixing, ϵ = sum of 4 other FSDD recordings	0.633
Reverb, $T_{60} = 0.6\,\mathrm{s}$	$y * \mathrm{rir}$: unit impulse at $\tau = 0$ plus a tail $0.3\,\epsilon[\tau]\,e^{-6.91\,\tau/(f_s T_{60})}$ (energy $-60\,\mathrm{dB}$ at T_{60}); rir normalised to unit energy	0.940
Telephone	zero-phase Butterworth band-pass 300–3400 Hz; then 8-bit μ-law codec: compress with $F(y) = \mathrm{sgn}(y)\frac{\ln(1+\mu_{\mathrm{law}}\lvert y\rvert)}{\ln(1+\mu_{\mathrm{law}})}$, $\mu_{\mathrm{law}} = 255$, quantise to 256 levels, expand with F^{-1}	0.733
Unseen speaker	train on 5 speakers, test on the 6th (each of the 6 in turn)	0.71–0.99

Model: $S = 3$, $M = 2$. $\overline{y^2}$: mean power of the signal. Reverb and telephone outputs are rescaled to the input power.

Your task: explain each drop with what the MFCC and the GMMs assume (which mel bands carry the energy of an 8 kHz digit? what does a convolution become after the log?).

Part 4

Language models measured alone

A language model is a probability over word sequences

Chain rule (exact), with an end token $w_{L+1} = \langle/\mathrm{s}\rangle$ so that sentences of every length share one distribution:

$$P(W) = \prod_{i=1}^{L+1} P(w_i \mid w_1, \ldots, w_{i-1}).$$

n-gram assumption: keep only the last $n-1$ tokens (a Markov chain of order $n-1$):

$$P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid h_i), \qquad h_i = (w_{i-n+1}, \ldots, w_{i-1}), \quad w_i = \langle\mathrm{s}\rangle \text{ for } i \le 0.$$

Maximum likelihood

$$P_{\mathrm{ML}}(w \mid h) = \frac{\#(h\,w)}{\#(h)}$$

$\#(h\,w)$: occurrences of h followed by w; $\#(h)$: occurrences of h as a history. The LM predicts tokens of $\mathcal{V}' = \mathcal{V} \cup \{\langle/\mathrm{s}\rangle\}$, never $\langle\mathrm{s}\rangle$.

Counting n-grams (`NGramLM.fit`)

Pad each sentence with $n-1$ tokens $\langle \mathrm{s} \rangle$ in front and one $\langle /\mathrm{s} \rangle$ at the end. Then, for each *predicted* position $i = 1, \ldots, L+1$ and each order $r = 1, \ldots, n$, with $h_i^{(r)}$ the last $r-1$ tokens before w_i (it may contain $\langle \mathrm{s} \rangle$):

$$\#\big(h_i^{(r)}\, w_i\big) \mathrel{+}= 1 \quad (\texttt{counts[r]}), \qquad \#\big(h_i^{(r)}\big) \mathrel{+}= 1 \quad (\texttt{context[r]}).$$

Keys are tuples; for $r = 1$ the history is the empty tuple `()`, so $\#(\,) =$ number of predicted tokens.

Example: $n = 2$, one sentence "1 2" $\to \langle \mathrm{s} \rangle$ 1 2 $\langle /\mathrm{s} \rangle$

Predicted w_i	**order 2**: `counts[2]` / `context[2]`	**order 1**: `counts[1]` / `context[1]`
1	$(\langle \mathrm{s} \rangle, 1)$ / $(\langle \mathrm{s} \rangle)$	(1) / $()$
2	$(1, 2)$ / (1)	(2) / $()$
$\langle /\mathrm{s} \rangle$	$(2, \langle /\mathrm{s} \rangle)$ / (2)	$(\langle /\mathrm{s} \rangle)$ / $()$ $\quad \Rightarrow \#(\,) = 3$

$\langle \mathrm{s} \rangle$ is never counted as a predicted token: otherwise the unigram no longer sums to one over $\mathcal{V}'$.

The catch: unseen n-grams get probability zero

Training text, bigrams ($n = 2$): “1 2 3” “1 2 4” “2 3 3”

$$P_{\mathrm{ML}}(3 \mid 2) = \frac{\#(2\,3)}{\#(2)} = \frac{2}{3}, \qquad P_{\mathrm{ML}}(5 \mid 2) = \frac{0}{3} = 0 \;\Rightarrow\; P(\text{“1 2 5”}) = 0.$$

One unseen event makes a whole sentence impossible, and $\log 0$ makes the perplexity infinite.

Add-k smoothing (default $k = 0.1$)

$$P_k(w \mid h) = \frac{\#(h\,w) + k}{\#(h) + k\,|\mathcal{V}'|}, \qquad \sum_{w \in \mathcal{V}'} P_k(w \mid h) = 1.$$

Every event gets at least $k/(\#(h) + k|\mathcal{V}'|)$. Simple, but too much mass on unseen events when $|\mathcal{V}'|$ is large.

Interpolation: back off to shorter histories

Linear interpolation (Jelinek–Mercer), as implemented in `NGramLM`

$$P_{\text{int}}(w \mid h) = \sum_{r=1}^{n} \eta_r \, P_k(w \mid h^{(r)}), \qquad h^{(r)} = \text{last } r{-}1 \text{ tokens of } h, \quad \eta_r \geq 0, \ \sum_r \eta_r = 1.$$

- Example, $n = 3$, words u, v, w:
  $P_{\text{int}}(w \mid u\,v) = \eta_3 \, P_k(w \mid u\,v) + \eta_2 \, P_k(w \mid v) + \eta_1 \, P_k(w)$, where $P_k(w)$ uses $\#(\,)$ = number of predicted training tokens.
- Code defaults for orders $n, n{-}1, \ldots, 1$: $(0.6,\ 0.3,\ 0.1,\ 0.05,\ 0.05)$, truncated to n values and renormalised.
- The weights η_r should be tuned on held-out text, never on the test text.

Each P_k already sums to one over $\mathcal{V}'$, so their convex combination does too.

What production systems use: Kneser–Ney (day 2, via KenLM)

You do not implement this; KenLM builds it on day 2. Two ideas, shown for a bigram:

- **Absolute discounting**: subtract a fixed discount $\gamma \in (0, 1)$ from every seen count; give the freed mass to the lower order.
- **Continuation probability**: the lower order counts *in how many different contexts* w appears, not how often.

$$P_{\mathrm{KN}}(w \mid u) = \frac{\max(\#(u\,w) - \gamma,\, 0)}{\#(u)} + \frac{\gamma \left|\{w' : \#(u\,w') > 0\}\right|}{\#(u)}\, P_{\mathrm{cont}}(w),$$

$$P_{\mathrm{cont}}(w) = \frac{\left|\{u' : \#(u'\,w) > 0\}\right|}{\left|\{(u', w') : \#(u'\,w') > 0\}\right|}.$$

Classic example: "Francisco" is frequent but follows almost only "San": its continuation probability is low, so it is rarely predicted after an unseen history.

Perplexity measures a language model on its own

On a test set $\mathcal{D}$ of sentences, with $K = \sum_{W \in \mathcal{D}} (L_W + 1)$ predicted tokens ($\langle/\mathrm{s}\rangle$ included):

$$\mathrm{PP} = \exp\Big(-\frac{1}{K} \sum_{W \in \mathcal{D}} \sum_{i=1}^{L_W+1} \log P(w_i \mid h_i) \Big).$$

Interpretation

- $\log \mathrm{PP}$: average negative log-probability per token (cross-entropy, in nats).
- Effective branching factor: if every $P(w_i \mid h_i) = 1/|\mathcal{V}'|$, then $\mathrm{PP} = |\mathcal{V}'|$ ($= 11$ here).
- Lower is better; comparable only on the *same test text and vocabulary*.

TODO `NGramLM` in `speech/ngram.py`

`fit`: counts of every order (slide 24).

`prob(w, history)`: P_k or P_{int}; history left-padded with $\langle\mathrm{s}\rangle$.

`perplexity`: the formula above, $\langle/\mathrm{s}\rangle$ counted in K.

Perplexity versus order, data size and smoothing

Synthetic "digit grammar": each digit depends on the previous *two*, $P(w_i \mid w_{i-2}, w_{i-1})$ drawn from a Dirichlet(0.3); sentences of 4–9 digits; 1 000 test sentences. (`--part lm`, seed 0.)

Training sentences, smoothing	$n=1$	$n=2$	$n=3$	$n=4$	$n=5$
50, add-k	10.75	11.15	**8.08**	8.61	8.63
50, interpolated	10.75	9.77	7.76	**7.73**	7.99
5 000, add-k	10.69	8.89	**5.05**	5.16	5.67
5 000, interpolated	10.69	9.14	5.73	**5.17**	5.23

- Add-k: the trigram matches the source; longer histories are too sparse. With 50 sentences even the bigram is worse than the unigram.
- Interpolated, default η: the trigram gives 40 % weight to orders 1–2, which cannot capture the source; $n=4$ gives 86 % to orders ≥ 3. Tuning η fixes this.
- Unigram ≈ 10.7: close to the uniform value $|\mathcal{V}'| = 11$.

The decoding graph grows with n (L2): the trigram wins on text, but costs V times more decoding states.

Lab, this morning

Implement (each TODO ↔ a slide)

TODO `pareto_mask_array` slide 7

TODO `recognise_isolated` slides 17, 21

TODO `NGramLM.fit, prob, perplexity` 23–28

Check: `pytest -m d1`

Run

```
export DSP_MILESTONE=d1
python -m speech.run_classical_baseline \
  --fsdd <dir> --part isolated
... --part lm
```

Decide, in `DECISIONS.md`

- Which acoustic configuration (S, M) under your latency budget? With which interval?
- Which order n and smoothing, for how much text?

Every experiment writes rows to `results/results.jsonl`; every choice gets a `DECISIONS.md` entry with its evidence (rows, figure).

Try your recogniser live

`python -m asr_demo.server`, open `http://localhost:8000`, pick an entry, hold Space, say **one digit**.

Menu entry	What it is	Works once
`digits/0-dtw-baseline`	stored examples + DTW, no model (provided)	at once
`digits/1-gmm`	one GMM per digit ($S = 1$, $M = 8$): frame order ignored	`recognise_isolated`
`digits/2-hmm`	one GMM-HMM per digit ($S = 5$, $M = 2$)	`recognise_isolated`

- An entry that needs your code names the function when you select it: implement it, restart the server. Models train on first use (10–25 s), then are cached.
- FSDD test files: DTW 0.95, GMM 0.99, HMM 0.99. Does frame order matter for *one* digit? For a *string* (this afternoon)?
- "Replay" runs the same recording on another entry: the only fair comparison.

Notation (1/3): evaluation, signal and features

Comparing systems

$\mathcal{M},\ \mathcal{M}'$	systems (configurations)
$f_o,\ o \in \mathcal{O}$	objectives, all "lower is better"
$\mathcal{M} \succ \mathcal{M}'$	$\mathcal{M}$ dominates $\mathcal{M}'$
$c_{\max}$	budget on a cost (e.g. latency)
N_{sys}	number of systems compared
$\hat{p},\ N_{\text{test}},\ z$	measured accuracy, test files, Gaussian quantile (1.96)
R	number of timed requests (p50, p90 over them)
RTF	compute time / audio duration
WER	(sub. + del. + ins.) / reference words

Degradations

$\epsilon,\ g$	noise signal, its gain
SNR	signal-to-noise ratio (dB)
rir, T_{60}	room impulse response, reverberation time
$F,\ \mu_{\text{law}}$	μ-law compander, $\mu_{\text{law}} = 255$

Signal and features

$y[\tau]$	waveform, sample index τ
f_{s}	sampling rate (8 kHz on FSDD)
$N_w,\ N_h$	window (200) and hop (80) in samples
$\text{win}[\tau]$	Hann window
$t,\ T$	frame index $1, \dots, T$ (10 ms hop)
f	FFT bin; $P_t[f]$ power spectrum
$\nu,\ N_{\text{mel}}$	mel band, number of bands (40)
$H_\nu[f]$	mel filter ν
$E_t[\nu],\ \tilde{E}_t[\nu]$	mel energy, its log (dB)
$\ell,\ \rho_\ell$	cepstral index, DCT scale
$c_t[\ell]$	MFCC ℓ of frame t, $\ell = 0, \dots, 12$
$\Delta c_t,\ \Delta^2 c_t$	1st and 2nd time derivatives (± 4 frames)
$\boldsymbol{x}_t \in \mathbb{R}^D$	feature vector of frame t, $D = 39$
x_d	component d of a vector $\boldsymbol{x}$
X	observation $(\boldsymbol{x}_1, \dots, \boldsymbol{x}_T)$
$\hat{\boldsymbol{\mu}},\ \hat{\boldsymbol{\sigma}}$	global normalisation statistics

Notation (2/3): acoustic models

HMM of one word w

θ_w	parameters $(\boldsymbol{\pi}, A, \{b_j\})$ of word w
$i,\ j$	states, $1, \ldots, S$ (S per word)
q_t	hidden state at frame t
π_j	$P(q_1 = j)$, initial probability
a_{ij}	$P(q_t = j \mid q_{t-1} = i)$, transition
κ	time spent in one state (frames)
$b_j(\boldsymbol{x})$	$p(\boldsymbol{x}_t = \boldsymbol{x} \mid q_t = j)$, emission density
$m,\ M$	Gaussian component, $1, \ldots, M$
$\omega_{jm}, \boldsymbol{\mu}_{jm}, \boldsymbol{\sigma}^2_{jm}$	weight, mean, diagonal variance
$\xi_m,\ \mathrm{logsumexp}$	arguments of a log-sum-exp

Algorithms

$\alpha_t(j)$	forward variable (sum over paths)
$\delta_t(j)$	Viterbi score (max over paths)
$\psi_t(j)$	back-pointer: best predecessor
$\hat{q}_t$	state of the best path
$\zeta_t(j),\ \zeta_t(i,j)$	EM posteriors $P(q_t{=}j \mid X)$, $P(q_{t-1}{=}i, q_t{=}j \mid X)$
$\zeta_t(m)$	responsibility of Gaussian m for frame t
$\theta^{\mathrm{old}},\ \theta^{\mathrm{new}}$	parameters before / after one EM iteration
Ω	score beam: keep δ within Ω of the best
N_{beam}	top-N beam: number of hypotheses kept

Word loop (L2)

$s \in \mathcal{S}$	a state of the loop; $\lvert\mathcal{S}\rvert$ (`self.S`)
$S_w,\ \mathrm{off}(w)$	states of word w, its offset
$\mathrm{first}(w),\ \mathrm{last}(w)$	entry and final state of word w
$\mathrm{word}(s)$	word that state s belongs to
$\mathcal{F}$	word-final states $\{\mathrm{last}(w)\}$
$\mathcal{E}$	entry transitions $\mathrm{last}(u) \to \mathrm{first}(v)$

Notation (3/3): words, language models, decoding

Words

$\mathcal{V},\ V$	vocabulary (10 digits), $V = \lvert\mathcal{V}\rvert$
$u,\ v,\ w$	words
W	word sequence $(w_1, \ldots, w_L)$, $\lvert W\rvert = L$
$\hat{W}$	recognised word sequence
$\mathcal{V}_{\text{sil}}$	$\mathcal{V} \cup \{\text{sil}\}$, with a silence word

Decoding

λ	LM weight (`lm_weight`)
β	word insertion term (`word_penalty`), $\beta < 0$
p_{sil}	probability of a silence between words
$\text{score}(W)$	$\log p(X \mid W) + \lambda \log P(W) + \beta\lvert W\rvert$
W^{ref}	reference transcription

n-gram language models

n	order: $n-1$ words of history
$h,\ h_i$	a history; the history of position i
$h^{(r)}$	last $r-1$ tokens of h
$\langle\text{s}\rangle,\ \langle/\text{s}\rangle$	start / end-of-sentence tokens
$\mathcal{V}'$	$\mathcal{V} \cup \{\langle/\text{s}\rangle\}$: what the LM can predict
$\#(\cdot)$	count in the training text
$P_{\text{ML}},\ P_k,\ P_{\text{int}}$	max. likelihood, add-k, interpolated
k	add-k smoothing constant
η_r	interpolation weight of order r
$P_{\text{KN}},\ P_{\text{cont}},\ \gamma$	Kneser–Ney, continuation prob., discount
$\mathcal{D},\ L_W$	test sentences; length of sentence W
K	number of predicted tokens in $\mathcal{D}$
PP	perplexity

All logarithms are natural (scores in nats). λ always denotes the LM weight; HMM parameters are θ.